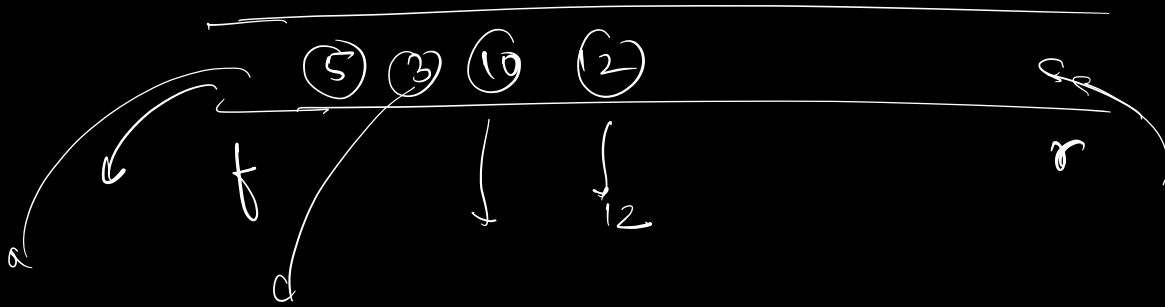


Queue

* Abstract data type

* FIFO $\rightarrow$ first in first out

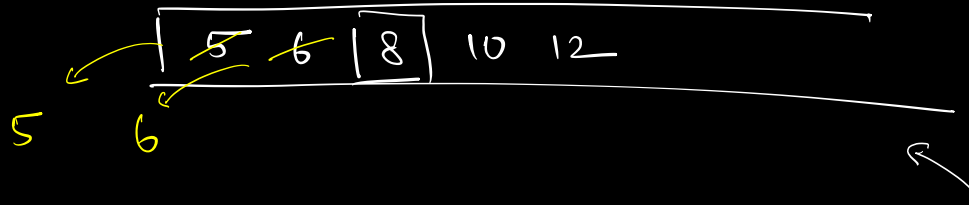


- Movie ticket counter
- Printer
- Call centres
- Job queues.

Operations by Queue:-

- $O(1)$
- 1) enqueue(x) $\rightarrow$ inserts x in rear end
 - 2) dequeue() $\rightarrow$ removes from front end and returns the data
 - 3) front() / peek() $\rightarrow$ returns the data from front
 - 4) isEmpty() $\rightarrow$

$E(5)$ $E(6)$ $E(8)$ $E(10)$ $D()$ $D()$ $E(12)$ $F()$
↓
8/11

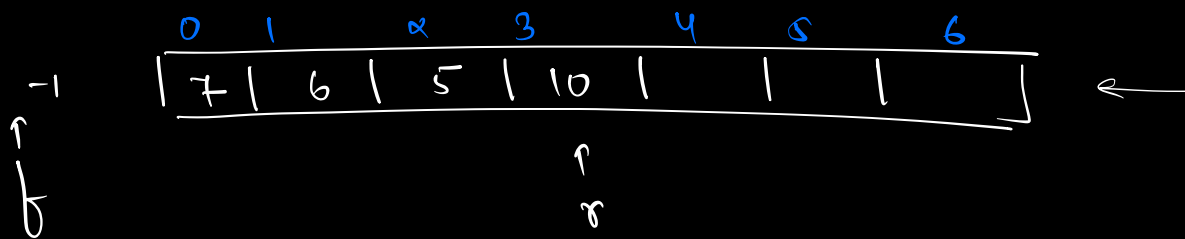


→ Implementation of queue:-

• array

$size = 0$ $f = -1$
 $r = -1$

capacity = 7



```

enqueue(x) {
    if (size == cap) {
        queue is full
    }

```

```

    r = (r + 1) % cap;
    arr[r] = x;
    size++;
}

```

```

front() {

```

```

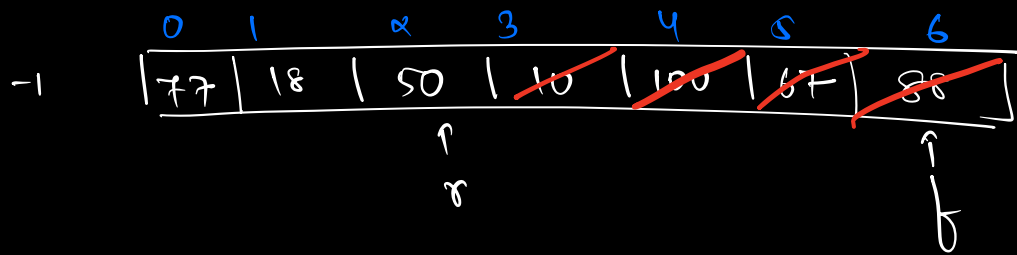
    if (size == 0) {
        queue is empty
    }

```

```

    return arr[(f + 1) % cap];
}

```



$queue \Rightarrow (f, r]$
 $\downarrow \quad \uparrow$
 exclusive inclusive

dequeue() {

 if (size == 0) {

 queue is empty

 }

 size--

$f = (f + 1) \% cap$; return array

}

→ Circular impl of a queue

• linked list impl of queue

enqueue(x) $\leftarrow$ insert at rear | end | tail
 $\downarrow$
tail ptr $\rightarrow O(1)$

dequeue() $\leftarrow$ remove from front | head
 $\downarrow$
 $O(1)$
head = head.next

front() $\leftarrow$ head.data

isEmpty $\leftarrow$ (head == null)

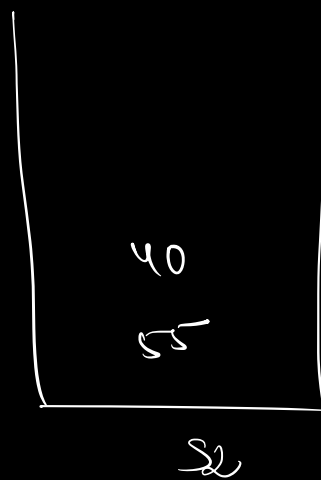
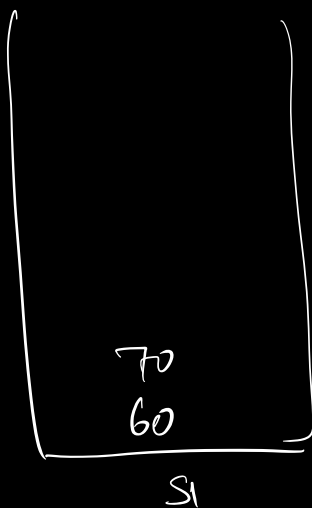
Q Implement queue using stack.

→ push / pop / top / isEmpty

↓
end
del
front
isEmpty

* queue impl is not possible using single stack

~~1~~ ~~9~~ ~~3~~ ~~6~~ ~~1~~ ~~8~~ ~~20~~ 40 55 60 70



4

9

3

6

1

8

20

```
enqueue(n) {
```

```
    s1.push(n)
```

```
}
```

```
dequeue() {
```

```
    if (s2.isEmpty()) {
```

```
        while (!s1.isEmpty()) {
```

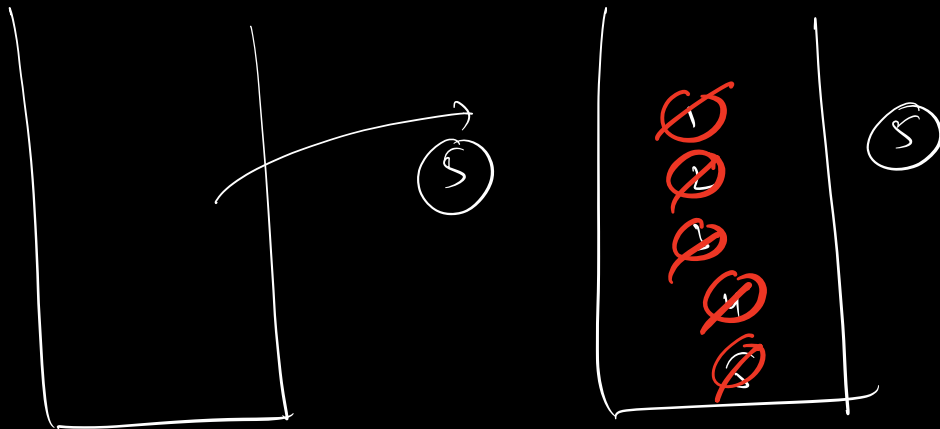
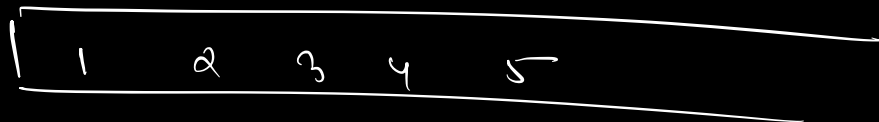
```
            s2.push(s1.pop())
```

```
        }
```

```
    }
```

```
    s2.pop()
```

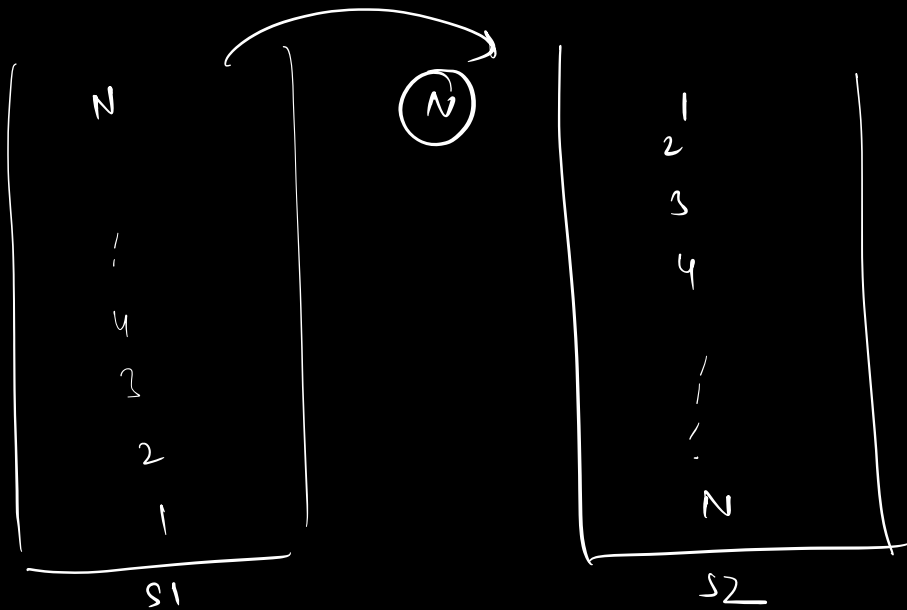
```
}
```



$$10 \rightarrow \underline{\underline{5}}$$

$$\text{avg} \rightarrow \text{pop} \rightarrow \alpha = \underline{\underline{O(1)}}$$

1 1 2 3 4 ... N

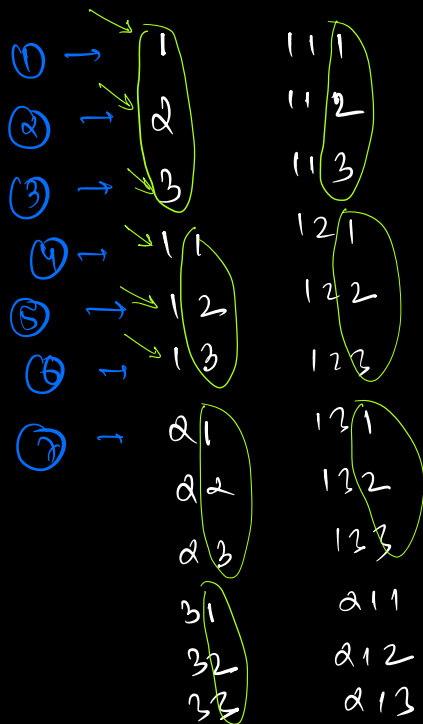


$$\frac{(N + N)}{N} = (2) \rightarrow \underline{\underline{O(1)}}$$

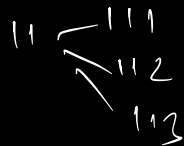
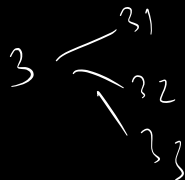
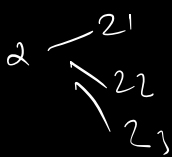
deque $\Rightarrow$ worst $O(N)$
 best $O(1)$
 amortised $\rightarrow \boxed{O(1)}$

enqueue $\Rightarrow O(1)$

Q2. Find N^{th} no. from pattern. [1 2 3]



12 13 21 22 23 111 112 113



[1 2 3]

TC $\approx O(N)$
SC $\approx O(N)$

Q 3. N^{th}

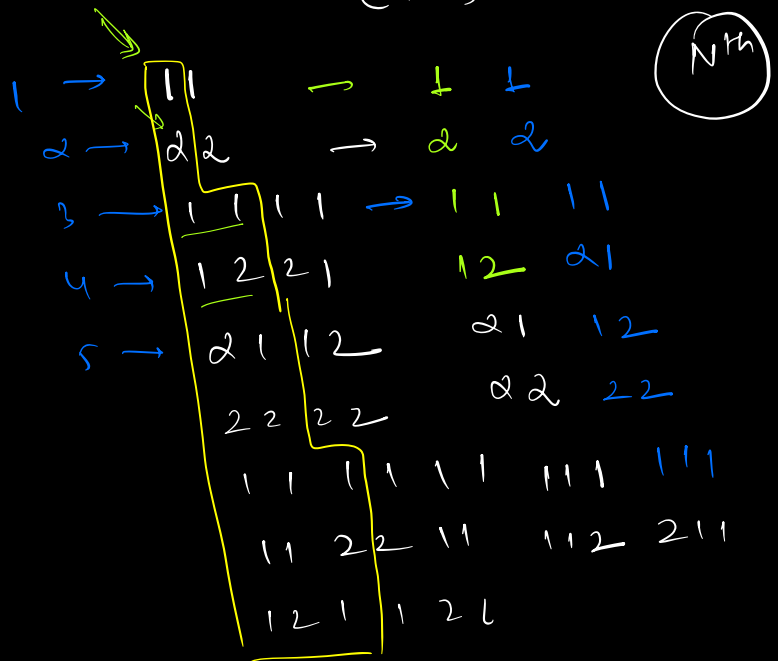
perfect no.

→ even length

→ palindrome

→ (1,2)

[1,2]



generate N^{th} no. from prev. question [1,2]

1 1

2 2

1 1 1 1

1 2 2 1

2 1 1 2

2 2 2 2

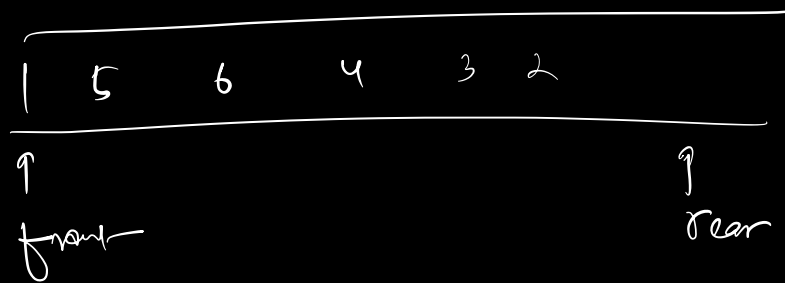
1 1 1 1 1 1

1 1 2 2 1 1

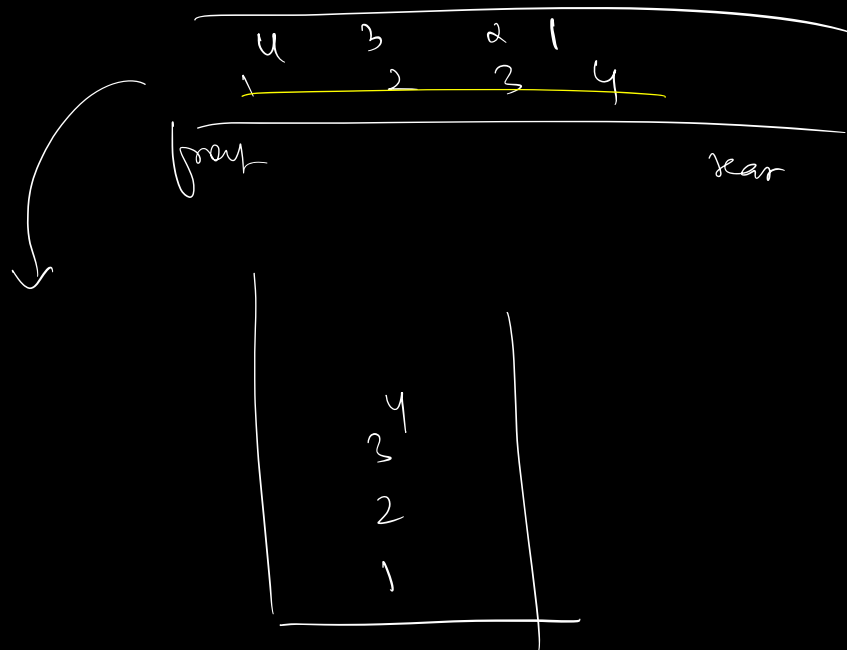
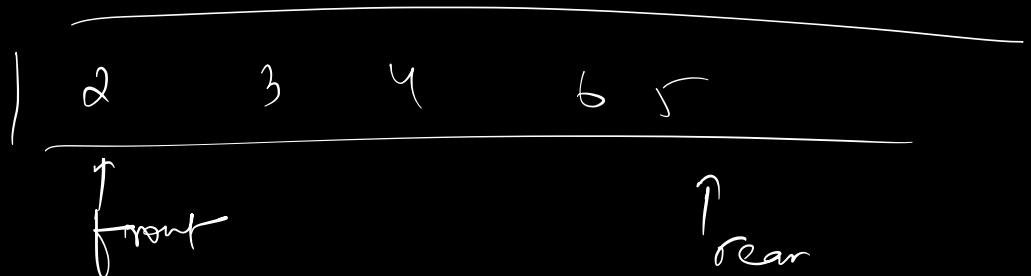
1 2 1 1 2 1

1 2 2 2 2 1

Q3. Reverse a queue.



Ans:



H.W. =

code to reverse a queue using stack